

KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Web Programlama

React II: Hooks, Form, Durum Paylaşımı

Hafta 5

React II: Hooks, Form, Durum Paylaşımı

Doç. Dr. Tolga Berber

KTÜ Fen Fakültesi, Bilgisayar Bilimleri Bölümü · https://avesis.ktu.edu.tr/tberber

Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim Neden Önemli?

Geçen hafta `Koltuk` bileşeni kendi seçim durumunu (`useState`) tutuyordu — ama `App` bunu hiç görmüyordu.

- Bir koltuğa tıklandığında sayfanın başka bir yerinde "toplam kaç koltuk seçili, toplam tutar ne" sorusuna cevap veremediniz — bilgi `Koltuk`'un içinde hapisti
- Sekme başlığı, bir geri sayım gibi "render'ın dışındaki" işler de hiç güncellenmiyordu — `useState` tek başına bunun için yetmez
- Bu hafta bu iki boşluğu dolduruyorsunuz: `useEffect` ile render dışı işler, `Context` ile paylaşılan durum — Etkinlik/Bilet Sisteminin ilk gerçek sepet katmanı bugün kuruluyor

YG II'nin `std::thread`'i başlatıp durdurma disiplinini biliyorsunuz — bugün aynı "temizleme" fikrini React'in `useEffect`'inde göreceksiniz.

Bu Hafta

◎ Öğrenme Hedefleri

Bu hafta sonunda şunları yapabileceksiniz:

1. `useEffect`'i yan etki, bağımlılık dizisi ve temizleme fonksiyonuyla açıklayıp bir başlık/geri sayım örneği uygulamak
2. `value`/`onChange` ile kontrollü girdi kurup form state'i, doğrulama ve `onSubmit` uygulamak
3. Prop drilling'i açıklayıp `createContext`/ `useContext` ile bir `SepetContext` kurmak
4. Tekrar eden mantığı özel hook'lara (`useSepet`, `useGeriSayim`) çıkarıp bileşen bölme kararı vermek
5. Bunları birleştirip Etkinlik/Bilet Sistemine ilk sepet katmanını eklemek

BÖLÜM 1

Ünite 1 — useEffect ve Yaşam Döngüsü

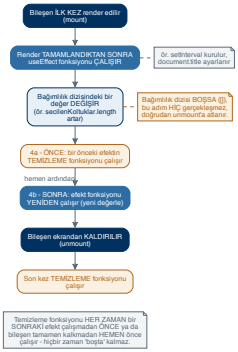
2 • İçerik Yan Etki, Bağımlılık Dizisi, Temizleme

- Yan etki: render işinin (JSX üretmenin) dışında dış dünyaya dokunan iş — belge başlığını değiştirmek, bir zamanlayıcı kurmak; render sırasında doğrudan yapılırsa öngörülemez tekrarlara yol açar
- `useEffect(() => { ... }, [bagimliliklar])`: fonksiyon render tamamlandıktan sonra çalışır; bağımlılık dizisi ne zaman tekrar çalışacağını belirler
- Üç hâl: dizi hiç verilmez → her render'dan sonra; `[]` (boş) → yalnız ilk render'dan sonra bir kez; `[deger]` → yalnız o değer değiştiyse
- Temizleme fonksiyonu: efektin geri döndürdüğü fonksiyon — unmount'tan hemen önce ve efekt tekrar çalışmadan hemen önce çağrılır

Tanım: Yan etki (*side effect*)

Bir bileşenin render işinin dışında, dış dünyaya (belge, zamanlayıcı, ağ) dokunan iştir.

`useEffect` Yaşam Döngüsü



Analoji: Vitrin Tabelası ve Toplantı Odası Işığı

Analoji

Yan etkiyi bir dükkânın vitrin tabelasını, dükkânın içindeki stok değiştiğinde ayrıca güncellemeye benzetin: tezgahtaki satış işlemi (render) bir şey, vitrindeki tabelayı değiştirmek (belge başlığı, zamanlayıcı) ayrı, ek bir iştir — biri diğerinin otomatik parçası değildir.

Analoji

Temizleme fonksiyonunu bir toplantı odasını kullandıktan sonra ışığı söndürüp kapıyı kilitlemeye benzetin: bir sonraki kullanım (yeni render ya da unmount) için ortamı temiz bırakırsınız — aksi halde ışık (zamanlayıcı) boş odada yanık kalır.

Kod: Belge Başlığı Efekti — `AppIcerik`

```
// Sekme başlığını seçili koltuk sayısına göre güncelleyen yan etki.
// secilenKoltuklar'ın NEREDEN geldiğini (SepetContext) durum
// kaldırma/Context ünitesinde göreceksiniz — bu efekt yalnızca
// useEffect'in kendisine (biçimine, bağımlılık dizisine) odaklanır.
useEffect(() => {
  document.title =
    secilenKoltuklar.length > 0
      ? `(${secilenKoltuklar.length}) KTÜ Etkinlik ve Bilet Sistemi`
      : 'KTÜ Etkinlik ve Bilet Sistemi';
}, [secilenKoltuklar.length]);
```

`secilenKoltuklar`'ın nereden geldiğini Ünite 3'te göreceksiniz — bu örnek yalnızca `useEffect`'in kendisine odaklanır.

Kod: Geri Sayım — `useGeriSayim.ts`

```ts
// baslangicSaniye'den geriye sayar: useState ile kalanSaniye tutulur,
// useEffect içindeki setInterval her saniye setKalanSaniye'yi çağırır.
// Süre bitince (bittiMi) zamanlayıcı YENİDEN kurulmaz; efekt her
// çalıştığında (ve bileşen ekrandan kalktığında) döndürdüğü temizleme
// fonksiyonuyla clearInterval çağrılır — bu olmasaydı bileşen unmount
// olsa bile zamanlayıcı arka planda çalışmaya devam ederdi.
export function useGeriSayim(baslangicSaniye: number) {
  const [kalanSaniye, setKalanSaniye] = useState(baslangicSaniye);
  const bittiMi = kalanSaniye <= 0;

  useEffect(() => {
    if (bittiMi) return;

    const id = setInterval(() => {
      setKalanSaniye((onceki) => (onceki <= 1 ? 0 : onceki - 1));
    }, 1000);

    return () => clearInterval(id);
  }, [bittiMi]);

  return { kalanSaniye, bittiMi };
}
```

Temizleme (`clearInterval`) olmasaydı bileşen unmount olsa bile zamanlayıcı arka planda çalışmaya devam ederdi.

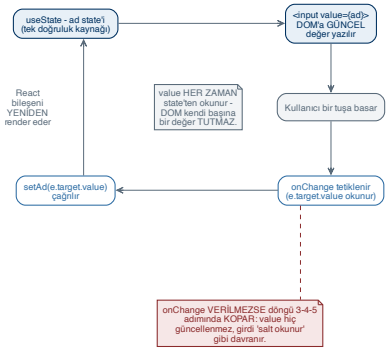
BÖLÜM 2

Ünite 2 — Formlar ve Kontrollü Girdi

2 • İçerik Kontrollü Girdi, Tiplenmiş Form State'i

- Kontrollü girdi: `<input>`'un `value`'su her zaman React state'inden okunur — kullanıcı yazınca `onChange` tetiklenir, state güncellenir, yeniden render `value`'yu yansıtır; tek doğruluk kaynağı state'tir, DOM değil
- `value` verip `onChange` vermemek yaygın bir hata — React "salt okunur girdi" uyarısı verir, kullanıcı hiçbir şey yazamaz
- Tüm alanlar tek bir `interface` ile gruplanır; bir alanı güncellerken diğerlerini korumak için yayma operatörü kullanılır: `{ ...form, ad: e.target.value }`
- `select`/`checkbox`: `checkbox` `value` yerine `checked`, `e.target.value` yerine `e.target.checked` kullanır
- `onSubmit` işleyicisinin ilk satırı neredeyse her zaman `e.preventDefault()` — aksi halde tarayıcı sayfayı yeniden yükler, tüm React state'i sıfırlanır

Kontrollü Girdi Döngüsü



Analoji: Gişe Memurunun Defteri

Analoji

Kontrollü girdiyi bir gişe memurunun, doldurduğunuz her formu anında bir deftere kopyalamasına benzetin: kâğıttaki (DOM'daki) yazı ile defterdeki (state'teki) kayıt her zaman aynı tutulur — memur (React) bu eşitliği `onChange` ile sağlar.

Kod: Form State'i ve Doğrulama — `BiletFormu.tsx`

```tsx
// Tüm alanlar TEK bir useState çağrısında tutulur; bir alanı
// güncellerken diğerlerini KORUMAK için yayma operatörü kullanılır
// (Hf.4'ün değişmezlik ilkesinin form state'ine uygulanışı).
const [form, setForm] = useState<BiletFormVeri>(BOS_FORM);
const [hata, setHata] = useState<string | null>(null);
const { secilenKoltuklar, sepetiTemizle } = useSepet();
```

`secilenKoltuklar`/`sepetiTemizle` `useSepet()`'ten gelir — Ünite 3-4'te açıklanacaktır.

Kod: onSubmit + preventDefault — Doğrulama (I)

```
39  function submitEt(e: FormEvent<HTMLFormElement>) {
40    e.preventDefault();
41
42    if (form.ad.trim() === '') {
43      setHata('Ad alanı boş olamaz.');
44      return;
45    }
46    if (secilenKoltuklar.length === 0) {
47      setHata('Sepette en az bir koltuk seçili olmalı.');
48      return;
49    }
```

İlk satır `e.preventDefault()` — aksi halde tarayıcı sayfayı yeniden yükler. Sonra ad ve koltuk seçimi koşulları kontrol edilir.

Kod: onSubmit + preventDefault — Doğrulama (II)

```
51    const satinAlmaZamani = new Date().toISOString();
52    const yeniBiletler: Bilet[] = secilenKoltuklar.map((koltuk) => ({
53      id: sonrakiBiletId++,
54      koltukId: koltuk.id,
55      kullaniciAdi: form.ad,
56      satinAlmaZamani,
57      durum: 'olusturuldu',
58    }));
59
60    onBiletlerOlustu(yeniBiletler);
61    sepetiTemizle();
62    setForm(BOS_FORM);
63    setHata(null);
64  }
```

Ad boşsa veya sepette hiç koltuk yoksa yukarıdaki koşullarda erken `return` ile `Bilet[]` oluşturulmaz; aksi halde her koltuk için bir `Bilet` üretilip sepet temizlenir.

BÖLÜM 3

Ünite 3 — Durum Kaldırma ve Context

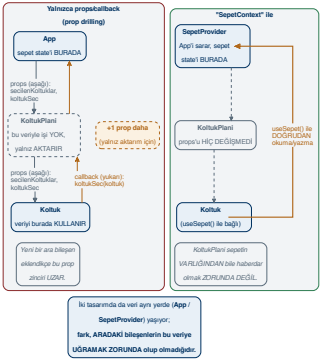
2 • İçerik Durum Kaldırma, Prop Drilling, `Context`

- `Koltuk` kendi `durum`'unu tutuyor ama `SepetOzeti`/ `BiletFormu` bu bilgiye ihtiyaç duyuyor — bilgi `Koltuk`'un dışından görünmüyor
- Durum kaldırma: paylaşılan durumu ortak en yakın üst bileşene taşımak; üst bileşen durumu tutar, alta güncel değeri ve bir callback verir
- Prop drilling: ara bileşenler (`KoltukPlani`) o veriyle hiçbir iş yapmasa bile props'una sadece alt'a aktarmak için eklemeniz gerekir — ağaç derinleştikçe katlanarak büyür
- `createContext`/`useContext`: ağacın herhangi üst noktasında yayınlanan, herhangi alt noktadan aradakilere uğramadan okunabilen bir veri kanalı

Tanım: Bağlam (*context*)

Ağacın herhangi bir üst noktasında sağlanan, herhangi bir alt noktadan aradaki bileşenlere uğramadan okunabilen veri kanalıdır.

Prop Drilling vs `Context`



Analoji: Zarfı Elden Uğratmak vs Bina Anonsu

Analoji

Prop drilling'i bir zarfı, ilgisi olmayan her ara kata elden uğratarak üst kata göndermeye benzetin: asansör (Context) yokken zarfı 5. kata göndermek için 2., 3. ve 4. kat sakinlerinin de zarfı eline alıp geçirmesi gerekir — hiçbiri içerikle ilgilenmez, yalnızca taşır.

Analoji

Context/Provider'ı bir binadaki ortak anons/santral sistemine benzetin: bina yönetimi (Provider) bir duyuruyu sisteme verir, hangi kattaki daire isterse (herhangi bir alt bileşen) doğrudan kendi hoparlöründen (`useContext`) dinler — duyuru kat kat elden ele dolaştırılmaz.

Kod: SepetContext Tanımı (createContext)

```tsx
// Başlangıç değeri null: bir bileşen SepetProvider'ın DIŞINDA
// useContext çağırırsa bunu ayırt edebilmek için (bkz. hooks/
    useSepet.ts).
export const SepetContext = createContext<SepetBaglami | null>(null);
```

Başlangıç değeri `null`: bir bileşen `SepetProvider`'ın dışında `useContext` çağırırsa bunu ayırt edebilmek için.

Kod: Koltuk Artık Context'ten Okuyor

```
// Koltuğun seçili olup olmadığı artık kendi yerel state'inde değil,
// ortak SepetContext'teki secilenKoltuklar dizisinde tutulur; App/
// KoltukPlani'nın haberi olmadan bu bileşen doğrudan Context'ten okur.
// Tıklama koltukSec'i çağırıp koltuğu sepete ekler/çıkarır; dolu bir
// koltuk (veri.ts'ten gelen kalıcı durum) hiç tıklanamaz.
const { secilenKoltuklar, koltukSec } = useSepet();
const seciliMi = secilenKoltuklar.some((k) => k.id === koltuk.id);
const durum = koltuk.durum === 'dolu' ? 'dolu' : seciliMi ? 'secili' : 'bos';

function tikla() {
  if (koltuk.durum === 'dolu') return;
  koltukSec(koltuk);
}
```

Yerel `useState` kaldırıldı; `App`/`KoltukPlani`'nın haberi bile olmadan ortak `secilenKoltuklar`'dan okunuyor.

BÖLÜM 4

Ünite 4 — Özel Hook’lar ve Kompozisyon

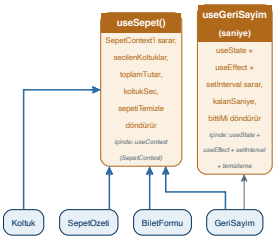
2 • İçerik Özel Hook, Kompozisyon, Bileşen Bölme Ölçütü

- Özel hook: adı `use` ile başlayan, içinde başka hook'lar çağıran sıradan bir fonksiyon — state tutabilir, efekt çalıştırabilir ama JSX döndürmez; yalnız mantık paylaşır
- `useSepet()`: `useContext(SepetContext)`'i sarar, `null` dönerse açık bir hata fırlatır — her bileşen kontrolü tekrar tekrar yazmaz
- `useGeriSayim()`: Ünite 1'in `useState`+`useEffect`+ `setInterval` deseni şimdi bir özel hook olarak adlandırılır
- Bileşen bölme sinyalleri: dosya okunmakta zorlandığınız kadar uzuyorsa, aynı mantık TEKRARLANIYORsa, iki ilgisiz sorumluluk bir arada ise

Tanım: Özel hook (*custom hook*)

Adı `use` ile başlayan, içinde başka hook'lar çağıran, mantık paylaşan (JSX döndürmeyen) fonksiyondur.

Özel Hook'ların Bileşenler Arasında Paylaşımı



Analoji: El Kitabı ve Banka Talimatnamesi

Analoji

Özel hook'u birden fazla resepsiyon masasının aynı standart kayıt prosedürünü tekrar tekrar öğrenmesi yerine, prosedürü tek bir el kitabına yazıp herkesin ona başvurmasına benzetin: her masa (bileşen) aynı işi (sepeti okuma, geri sayım tutma) yeniden icat etmez, ortak el kitabını (hook'u) çağırır.

İpucu

`useReducer`, sepetteki eylemleri (`EKLE`/`CIKAR`/`TEMIZLE`) tek bir `sepetReducer` fonksiyonunda toplayan, `useState`'e alternatif bir durum yönetim biçimidir — birden çok ilişkili eylem olduğunda tercih edilebilir; bu haftanın çalışan kodu sadelik için `useState` kullanmaya devam eder.

Kod: useSepet Özel Hook'u

```
// useContext(SepetContext)'i sarar; her bileşenin kendi başına
// useContext + null kontrolü yazması yerine yalnızca useSepet() çağırması
// yeter. baglam null dönerse (SepetProvider dışında kullanılmışsa)
// belirsiz bir çalışma zamanı hatası yerine AÇIK, anlaşılır bir hata
// fırlatır.
export function useSepet() {
  const baglam = useContext(SepetContext);
  if (!baglam) {
    throw new Error('useSepet() yalnızca SepetProvider içinde kullanılabilir.');
  }
  return baglam;
}
```

Açık `throw` sayesinde yanlış kullanım (`SepetProvider` dışında) belirsiz bir hatayla değil, net bir mesajla hemen ortaya çıkar.

Kod: App/AppIcerik Ayrımı

```
// App artık yalnızca hangi etkinliğin bilet fiyatını kullanacağını
// belirleyip SepetProvider'ı kurar; asıl sayfa içeriği AppIcerik'tedir.
// AppIcerik, SepetProvider'ın İÇİNDE (bir alt bileşen olarak) durmak
// ZORUNDADIR — bir bileşen kendi sağladığı Context'i doğrudan okuyamaz,
// yalnızca alt'ları okuyabilir.
export function App() {
  return (
    <SepetProvider biletFiyati={AKTIF_ETKINLIK.biletFiyati}>
      <AppIcerik />
    </SepetProvider>
  );
}
```

`AppIcerik`, `SepetProvider`'ın içinde durmak zorundadır — bir bileşen kendi sağladığı Context'i doğrudan okuyamaz, yalnızca alt'ları okuyabilir.

BÖLÜM 5

Özet

Özet

Özet

- Yan etki + `useEffect` + bağımlılık dizisi + temizleme fonksiyonu: belge başlığı ve geri sayım örnekleriyle render dışı işleri yönettiniz
- Kontrollü girdi + tiplenmiş form state'i + `select`/ `checkbox` + doğrulama + `preventDefault`: `BiletFormu` bileşenini oluşturdunuz
- Durum kaldırma + prop drilling + `createContext`/ `useContext`: `SepetContext` ile paylaşılan bir durum kurdunuz
- Özel hook'lar (`useSepet`, `useGeriSayim`) + bileşen bölme ölçütü: tekrar eden mantığı tek bir yerde topladınız

5 • İzleme Aktarım — Başka Nerede?

- Bugün kurduğunuz `SepetContext`, Hf.10'da bir `OturumContext` (giriş yapmış kullanıcı bilgisi) ile aynı kalıpla genişleyecek — "birden çok bileşenin ihtiyaç duyduğu ortak durum" sorusunun cevabı bundan sonra her yerde Context olacak
- `useEffect`/temizleme deseni Hf.7'de gerçek veri çekmede, Hf.13'te bir WebSocket bağlantısı açıp kapatırken aynen karşınıza çıkacak
- `useSepet`/`useGeriSayim` kalıbı, Hf.7'nin `useEtkinlikler()` ve Hf.10'un `useOturum()` hook'larının doğrudan atasıdır
- Bugün oluşturduğunuz `Bilet[]` yalnızca `App`'in yerel belleğinde yaşıyor — sayfa yenilenince kaybolur; Hf.6'da bu veriyi bir sunucuya (FastAPI) taşımaya başlayacaksınız

Sırada Ne Var?

Bu hafta lab'da

Hafta 4'ün `istemci/` projesi üzerine `SepetContext`, `useSepet`, `useGeriSayim` ve `BiletFormu`'yu kendi elinizle yazıp Etkinlik/Bilet Sisteminin ilk gerçek sepet katmanını `npm run dev` ile tarayıcıda canlı test edeceksiniz.

- Kendinizi Sınayın her üniteyi kapsıyor: `[]` ile ikinci argümanın hiç verilmemesi arasındaki fark, `value` verip `onChange` vermemenin sonucu, `Context` olmadan prop drilling neden büyür, `useSepet`'teki `throw` satırını silerseniz ne olur
- Gelecek hafta: FastAPI I — HTTP'yi Koda Dökmek — bu haftanın yerel `Bilet[]`'i bir sunucuya taşınacak